PYTHON WEEK 5 TEST

Sync vs Async vs Multithreading (Interview Focus)

Instructions: (1) Solve all questions. (2) For output questions, write exact output. (3) For MCQ, multiple options may be correct; select all correct options. (4) Total: 100 | Sync: 20 | Async: 40 | Multithreaded: 40 | Output: 60 | MCQ: 40.

Part A: Predict the Output

Q1. Predict output for RLock re-entrant acquisition by same thread.

```
import threading

lock = threading.RLock()
value = 0

def fn():
    global value
    with lock:
        value += 1
        with lock:
            value += 2

fn()
print(value)
```

Answer:

Q2. Predict output for sync timeout fallback path.

```
def call(timeout):
    if timeout < 3:
        raise TimeoutError('slow')
    return 'ok'

try:
    print(call(3))
except TimeoutError:
    print('fallback')
finally:
    print('cleanup')
```

Answer:

Q3. Predict output for create_task concurrency and await order.

```
import asyncio

async def work(name, delay):
    await asyncio.sleep(delay)
    return f'{name}-done'

async def main():
    t1 = asyncio.create_task(work('A', 0.02))
    t2 = asyncio.create_task(work('B', 0.05))
    r1 = await t1
    r2 = await t2
    print(r1, r2)

asyncio.run(main())
```

Answer:

Q4. Predict output for call stack order in sync functions.

```
def a():
    print('a1')
    b()
    print('a2')

def b():
    print('b1')
    c()
    print('b2')

def c():
    print('c')

a()
```

Answer:

Q5. Predict output for start() vs run() thread behavior trap.

```
import threading

def task():
    print('from', threading.current_thread().name)

t = threading.Thread(target=task, name='W1')
t.run()
t.start()
t.join()
```

Answer:

Q6. Predict output for basic ThreadPoolExecutor map behavior.

```
from concurrent.futures import ThreadPoolExecutor

def sq(x):
    return x * x

with ThreadPoolExecutor(max_workers=4) as pool:
    print(list(pool.map(sq, [1, 2, 3, 4])))
```

Answer:

Q7. Predict output for asyncio.gather result ordering.

```
import asyncio

async def f(x, d):
    await asyncio.sleep(d)
    return x * 10

async def main():
    out = await asyncio.gather(f(1, 0.04), f(2, 0.02))
    print(out)

asyncio.run(main())
```

Answer:

Q8. Predict output for basic ThreadPoolExecutor map behavior.

```
from concurrent.futures import ThreadPoolExecutor

def sq(x):
    return x * x

with ThreadPoolExecutor(max_workers=3) as pool:
    print(list(pool.map(sq, [1, 2, 3, 4])))
```

Answer:

Q9. Predict output for semaphore-limited async task completion shape.

```
import asyncio

async def bounded(x, sem):
    async with sem:
        await asyncio.sleep(0.01)
        return x * 2

async def main():
    sem = asyncio.Semaphore(3)
    tasks = [bounded(i, sem) for i in range(5)]
    print(await asyncio.gather(*tasks))

asyncio.run(main())
```

Answer:

Q10. Predict output for sync retry loop with break/else.

```
for attempt in range(1, 4):
    if attempt == 3:
        print('ok', attempt)
        break
    print('retry', attempt)
else:
    print('failed')
```

Answer:

Q11. Predict output for lock-protected shared counter update.

```
import threading

counter = 0
lock = threading.Lock()

def inc(n):
    global counter
    for _ in range(n):
        with lock:
            counter += 1

n = 3591
t1 = threading.Thread(target=inc, args=(n,))
t2 = threading.Thread(target=inc, args=(n,))
t1.start(); t2.start()
t1.join(); t2.join()
print(counter)
```

Answer:

Q12. Predict output for semaphore-bounded thread section accounting.

```
import threading

sem = threading.Semaphore(2)
active = 0
max_active = 0
mu = threading.Lock()

def worker():
    global active, max_active
    with sem:
        with mu:
            active += 1
            max_active = max(max_active, active)
        with mu:
            active -= 1

threads = [threading.Thread(target=worker) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print(max_active)
```

Answer:

Q13. Predict output for producer-consumer using asyncio.Queue sentinel.

```
import asyncio

async def producer(q):
    for i in [1, 2, 3]:
        await q.put(i)
    await q.put(None)

async def consumer(q):
    out = []
    while True:
        item = await q.get()
        if item is None:
            q.task_done()
            break
        out.append(item * 10)
        q.task_done()
    return out

async def main():
    q = asyncio.Queue(maxsize=2)
    c = asyncio.create_task(consumer(q))
    await producer(q)
    await q.join()
    print(await c)

asyncio.run(main())
```

Answer:

Q14. Predict output for Event synchronization pattern.

```
import threading

evt = threading.Event()
state = []

def waiter():
    evt.wait()
    state.append('go')

t = threading.Thread(target=waiter)
t.start()
state.append('ready')
evt.set()
t.join()
print(state)
```

Answer:

Q15. Predict output for cancellation propagation in awaited task.

```
import asyncio

async def job():
    try:
        await asyncio.sleep(1)
    except asyncio.CancelledError:
        print('job-cancelled')
        raise

async def main():
    t = asyncio.create_task(job())
    await asyncio.sleep(0)
    t.cancel()
    try:
        await t
    except asyncio.CancelledError:
        print('main-cancelled')

asyncio.run(main())
```

Answer:

Q16. Predict output for asyncio.wait_for timeout handling.

```
import asyncio

async def slow():
    await asyncio.sleep(0.05)
    return 'done'

async def main():
    try:
        print(await asyncio.wait_for(slow(), timeout=0.02))
    except TimeoutError:
        print('timeout')

asyncio.run(main())
```

Answer:

Q17. Predict output for TaskGroup structured concurrency success case.

```
import asyncio

async def worker(name, d):
    await asyncio.sleep(d)
    return f'{name}:{d}'

async def main():
    results = []
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(worker('A', 0.01))
        t2 = tg.create_task(worker('B', 0.02))
    results.append(t1.result())
    results.append(t2.result())
    print(results)

asyncio.run(main())
```

Answer:

Q18. Predict output for asyncio.wait_for timeout handling.

```
import asyncio

async def slow():
    await asyncio.sleep(0.06)
    return 'done'

async def main():
    try:
        print(await asyncio.wait_for(slow(), timeout=0.03))
    except TimeoutError:
        print('timeout')

asyncio.run(main())
```

Answer:

Q19. Predict output for sync retry loop with break/else.

```
for attempt in range(1, 4):
    if attempt == 2:
        print('ok', attempt)
        break
    print('retry', attempt)
else:
    print('failed')
```

Answer:

Q20. Predict output for daemon flag configuration and join.

```
import threading

def work():
    print('work')

t = threading.Thread(target=work, daemon=True)
print(t.daemon)
t.start()
t.join()
print('done')
```

Answer:

Q21. Predict output for CPU-bound vs I/O-bound decision helper.

```
cpu_wait = 64
io_wait = 39
if io_wait > cpu_wait:
    model = 'async-or-threads'
elif cpu_wait > io_wait:
    model = 'process-or-optimized-sync'
else:
    model = 'mixed'
print(model)
```

Answer:

Q22. Predict output for asyncio.timeout context manager.

```
import asyncio

async def maybe_slow():
    await asyncio.sleep(0.06)
    return 'ok'

async def main():
    try:
        async with asyncio.timeout(0.04):
            print(await maybe_slow())
    except TimeoutError:
        print('ctx-timeout')

asyncio.run(main())
```

Answer:

Q23. Predict output for sync error categorization and retry decision.

```
def classify(code):
    if code >= 500:
        return 'retry'
    if code >= 400:
        return 'no-retry'
    return 'ok'

status = 503
action = classify(status)
print(status, action)
```

Answer:

Q24. Predict output for Future exception handling pattern.

```
from concurrent.futures import ThreadPoolExecutor

def work(x):
    if x == 0:
        raise ValueError('bad')
    return 10 // x

with ThreadPoolExecutor(max_workers=2) as pool:
    futures = [pool.submit(work, x) for x in [6, 0, 10]]
    out = []
    for f in futures:
        try:
            out.append(f.result())
        except ValueError:
            out.append('err')
print(out)
```

Answer:

Q25. Predict output for deadlock-avoidance lock-order helper.

```
import threading

a = threading.Lock()
b = threading.Lock()

def with_two(l1, l2):
    first, second = sorted((l1, l2), key=id)
    with first:
        with second:
            return 'ok'

print(with_two(a, b))
print(with_two(b, a))
print(112)
```

Answer:

Q26. Predict output for Future exception handling pattern.

```
from concurrent.futures import ThreadPoolExecutor

def work(x):
    if x == 0:
        raise ValueError('bad')
    return 10 // x

with ThreadPoolExecutor(max_workers=2) as pool:
    futures = [pool.submit(work, x) for x in [2, 0, 7]]
    out = []
    for f in futures:
        try:
            out.append(f.result())
        except ValueError:
            out.append('err')
print(out)
```

Answer:

Q27. Predict output for async design checklist scoring helper.

```
checks = {
    'timeout': True,
    'retry': True,
    'cancellation': False,
    'backpressure': True,
}
weight = 5
score = sum(weight for ok in checks.values() if ok)
print(score)
print('pass' if score >= weight * 3 else 'fix')
```

Answer:

Q28. Predict output for deterministic sync fallback chain.

```
def primary():
    raise ConnectionError

def secondary():
    return 3 + 7

try:
    print(primary())
except ConnectionError:
    print('using-secondary')
    print(secondary())
```

Answer:

Q29. Predict output for sync error categorization and retry decision.

```
def classify(code):
    if code >= 500:
        return 'retry'
    if code >= 400:
        return 'no-retry'
    return 'ok'

status = 500
action = classify(status)
print(status, action)
```

Answer:

Q30. Predict output for asyncio.shield protecting inner critical operation.

```
import asyncio

async def commit():
    await asyncio.sleep(0.03)
    return 'committed'

async def main():
    task = asyncio.create_task(commit())
    try:
        async with asyncio.timeout(0.01):
            await asyncio.shield(task)
    except TimeoutError:
        print('outer-timeout')
    print(await task)

asyncio.run(main())
```

Answer:

Q31. Predict output for asyncio.timeout context manager.

```
import asyncio

async def maybe_slow():
    await asyncio.sleep(0.06)
    return 'ok'

async def main():
    try:
        async with asyncio.timeout(0.02):
            print(await maybe_slow())
    except TimeoutError:
        print('ctx-timeout')

asyncio.run(main())
```

Answer:

Q32. Predict output for interview-style model selection helper.

```
workload = {'io': 57, 'cpu': 35, 'blocking_sdk': False}
if workload['io'] > workload['cpu'] and not workload['blocking_sdk']:
    pick = 'async'
elif workload['io'] > workload['cpu']:
    pick = 'threads-or-async+offload'
else:
    pick = 'sync-or-process'
print(pick)
```

Answer:

Q33. Predict output for submit + as_completed with deterministic post-processing.

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def job(x):
    return x * 10

with ThreadPoolExecutor(max_workers=4) as pool:
    futures = [pool.submit(job, i) for i in [3, 1, 2]]
    out = [f.result() for f in as_completed(futures)]
print(sorted(out))
```

Answer:

Q34. Predict output for mixed async + ThreadPoolExecutor integration.

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

def blocking(x):
    return x * 4

async def main():
    loop = asyncio.get_running_loop()
    with ThreadPoolExecutor(max_workers=2) as pool:
        tasks = [loop.run_in_executor(pool, blocking, i) for i in [1, 2]]
        out = await asyncio.gather(*tasks)
    print(out)

asyncio.run(main())
```

Answer:

Q35. Predict output for graceful shutdown with queue sentinel.

```
import queue
import threading

q = queue.Queue()
processed = []

def worker():
    while True:
        item = q.get()
        if item is None:
            q.task_done()
            break
        processed.append(item + 1)
        q.task_done()

t = threading.Thread(target=worker)
t.start()
for x in [10, 14]:
    q.put(x)
q.put(None)
q.join(); t.join()
print(processed)
```

Answer:

Q36. Predict output for TaskGroup ensuring child completion before exit.

```
import asyncio

async def child(i):
    await asyncio.sleep(0.01 * i)
    return i

async def main():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(child(1))
        t2 = tg.create_task(child(2))
    print(t1.result() + t2.result())

asyncio.run(main())
```

Answer:

Q37. Predict output for mixed async + blocking style with to_thread fanout.

```
import asyncio

def blocking_io(x):
    return x + 100

async def main():
    tasks = [asyncio.to_thread(blocking_io, i) for i in [1, 2, 3]]
    print(await asyncio.gather(*tasks))

asyncio.run(main())
```

Answer:

Q38. Predict output for mixed async + ThreadPoolExecutor integration.

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

def blocking(x):
    return x * 5

async def main():
    loop = asyncio.get_running_loop()
    with ThreadPoolExecutor(max_workers=2) as pool:
        tasks = [loop.run_in_executor(pool, blocking, i) for i in [1, 3, 4]]
        out = await asyncio.gather(*tasks)
    print(out)

asyncio.run(main())
```

Answer:

Q39. Predict output for sync critical-path analysis approximation.

```
stages = [9, 5, 10, 15, 13]
critical = []
for idx, ms in enumerate(stages):
    if ms > 8:
        critical.append((idx, ms))
print(critical)
print(len(critical))
```

Answer:

Q40. Predict output for gather exception handling with return_exceptions.

```
import asyncio

async def ok(delay):
    await asyncio.sleep(delay)
    return 'ok'

async def fail(delay):
    await asyncio.sleep(delay)
    raise ValueError('bad')

async def main():
    res = await asyncio.gather(ok(0.01), fail(0.04), return_exceptions=True)
    print(type(res[1]).__name__, res[0])

asyncio.run(main())
```

Answer:

Q41. Predict output for condition-based handoff between producer and consumer.

```
import threading

cond = threading.Condition()
ready = False
events = []

def consumer():
    global ready
    with cond:
        while not ready:
            cond.wait()
        events.append('consume')

def producer():
    global ready
    with cond:
        ready = True
        events.append('produce')
        cond.notify()

tc = threading.Thread(target=consumer)
tp = threading.Thread(target=producer)
tc.start(); tp.start()
tc.join(); tp.join()
print(sorted(events))
print('m', 9)
```

Answer:

Q42. Predict output for gather exception handling with return_exceptions.

```
import asyncio

async def ok(delay):
    await asyncio.sleep(delay)
    return 'ok'

async def fail(delay):
    await asyncio.sleep(delay)
    raise ValueError('bad')

async def main():
    res = await asyncio.gather(ok(0.02), fail(0.05), return_exceptions=True)
    print(type(res[1]).__name__, res[0])

asyncio.run(main())
```

Answer:

Q43. Predict output for mixed async + ThreadPoolExecutor integration.

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

def blocking(x):
    return x * 4

async def main():
    loop = asyncio.get_running_loop()
    with ThreadPoolExecutor(max_workers=2) as pool:
        tasks = [loop.run_in_executor(pool, blocking, i) for i in [1, 2, 3]]
        out = await asyncio.gather(*tasks)
    print(out)

asyncio.run(main())
```

Answer:

Q44. Predict output for create_task naming and introspection.

```
import asyncio

async def work():
    await asyncio.sleep(0)
    return 42

async def main():
    t = asyncio.create_task(work(), name='fetch-42')
    print(t.get_name())
    print(await t)

asyncio.run(main())
```

Answer:

Q45. Predict output for deterministic sync fallback chain.

```
def primary():
    raise ConnectionError

def secondary():
    return 3 + 6

try:
    print(primary())
except ConnectionError:
    print('using-secondary')
    print(secondary())
```

Answer:

Q46. Predict output for async design checklist scoring helper.

```
checks = {
    'timeout': True,
    'retry': True,
    'cancellation': False,
    'backpressure': True,
}
weight = 2
score = sum(weight for ok in checks.values() if ok)
print(score)
print('pass' if score >= weight * 3 else 'fix')
```

Answer:

Q47. Predict output for async design checklist scoring helper.

```
checks = {
    'timeout': True,
    'retry': True,
    'cancellation': False,
    'backpressure': True,
}
weight = 3
score = sum(weight for ok in checks.values() if ok)
print(score)
print('pass' if score >= weight * 3 else 'fix')
```

Answer:

Q48. Predict output for bounded queue backpressure signal handling.

```
import asyncio

async def main():
    q = asyncio.Queue(maxsize=1)
    await q.put('first')
    print(q.full())
    print(await q.get())
    q.task_done()
    print(q.empty())

asyncio.run(main())
```

Answer:

Q49. Predict output for cancellation-safe cleanup with finally in coroutine.

```
import asyncio

async def worker():
    try:
        await asyncio.sleep(1)
    finally:
        print('cleanup')

async def main():
    t = asyncio.create_task(worker())
    await asyncio.sleep(0)
    t.cancel()
    try:
        await t
    except asyncio.CancelledError:
        print('cancelled')

asyncio.run(main())
```

Answer:

Q50. Predict output for lock granularity effect simulation.

```
segments = [4, 7, 3, 10]
critical = [x for x in segments if x >= 5]
print(sum(critical))
print(len(critical))
```

Answer:

Q51. Predict output for asyncio.timeout context manager.

```
import asyncio

async def maybe_slow():
    await asyncio.sleep(0.06)
    return 'ok'

async def main():
    try:
        async with asyncio.timeout(0.03):
            print(await maybe_slow())
    except TimeoutError:
        print('ctx-timeout')

asyncio.run(main())
```

Answer:

Q52. Predict output for async design checklist scoring helper.

```
checks = {
    'timeout': True,
    'retry': True,
    'cancellation': False,
    'backpressure': True,
}
weight = 6
score = sum(weight for ok in checks.values() if ok)
print(score)
print('pass' if score >= weight * 3 else 'fix')
```

Answer:

Q53. Predict output for async lock around shared mutable state.

```
import asyncio

counter = 0
lock = asyncio.Lock()

async def inc_many(n):
    global counter
    for _ in range(n):
        async with lock:
            tmp = counter
            await asyncio.sleep(0)
            counter = tmp + 1

async def main():
    await asyncio.gather(inc_many(30), inc_many(30), inc_many(40))
    print(counter)

asyncio.run(main())
```

Answer:

Q54. Predict output for CPU-bound vs I/O-bound decision helper.

```
cpu_wait = 52
io_wait = 39
if io_wait > cpu_wait:
    model = 'async-or-threads'
elif cpu_wait > io_wait:
    model = 'process-or-optimized-sync'
else:
    model = 'mixed'
print(model)
```

Answer:

Q55. Predict output for deadlock-avoidance lock-order helper.

```
import threading

a = threading.Lock()
b = threading.Lock()

def with_two(l1, l2):
    first, second = sorted((l1, l2), key=id)
    with first:
        with second:
            return 'ok'

print(with_two(a, b))
print(with_two(b, a))
print(115)
```

Answer:

Q56. Predict output for sync design with small pure functions and composition.

```
def parse(x):
    return x.strip().upper()

def validate(x):
    return x.isalpha()

items = [' a ', 'b', 'c1', 'd'][:4]
clean = [parse(i) for i in items]
valid = [x for x in clean if validate(x)]
print(clean)
print(valid)
```

Answer:

Q57. Predict output for graceful shutdown with queue sentinel.

```
import queue
import threading

q = queue.Queue()
processed = []

def worker():
    while True:
        item = q.get()
        if item is None:
            q.task_done()
            break
        processed.append(item + 1)
        q.task_done()

t = threading.Thread(target=worker)
t.start()
for x in [9, 17]:
    q.put(x)
q.put(None)
q.join(); t.join()
print(processed)
```

Answer:

Q58. Predict output for condition-based handoff between producer and consumer.

```
import threading

cond = threading.Condition()
ready = False
events = []

def consumer():
    global ready
    with cond:
        while not ready:
            cond.wait()
        events.append('consume')

def producer():
    global ready
    with cond:
        ready = True
        events.append('produce')
        cond.notify()

tc = threading.Thread(target=consumer)
tp = threading.Thread(target=producer)
tc.start(); tp.start()
tc.join(); tp.join()
print(sorted(events))
print('m', 5)
```

Answer:

Q59. Predict output for thread pool sizing against external limit hint.

```
external_limit = 7
cpu = 2
workers = min(external_limit, cpu * 2)
print(workers)
print('bounded')
```

Answer:

Q60. Predict output for submit + as_completed with deterministic post-processing.

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def job(x):
    return x * 10

with ThreadPoolExecutor(max_workers=2) as pool:
    futures = [pool.submit(job, i) for i in [3, 1, 2]]
    out = [f.result() for f in as_completed(futures)]
print(sorted(out))
```

Answer:

Part B: MCQ (Multiple Correct; Select All That Apply)

Q61. `RLock` compared to `Lock`: (Select all that apply.)

- **A) Slower sorting**
- **B) Allows same thread to acquire recursively**
- **C) Works only in async**
- **D) Prevents all deadlocks automatically**

Q62. Threads are still very useful in Python for: (Select all that apply.)

- **A) I/O-bound concurrency**
- **B) Only sorting lists**
- **C) Replacing async always**
- **D) Eliminating locks**

Q63. CPU-bound task in default CPython generally benefits most from: (Select all that apply.)

- **A) Blindly adding threads**
- **B) Better algorithms / native acceleration / multiprocessing**
- **C) Removing all exceptions**
- **D) Queue only**

Q64. `asyncio.create_task(coro)` primarily: (Select all that apply.)

- **A) Runs coro in new process**
- **B) Schedules coroutine concurrently**
- **C) Makes coroutine synchronous**
- **D) Deep copies state**

Q65. Coroutine vs Task: task is: (Select all that apply.)

A) Raw function object

B) Scheduled wrapper managed by event loop

C) Multiprocessing worker

D) Lock primitive

Q66. Event loop is best described as: (Select all that apply.)

A) Thread pool manager only

B) Cooperative scheduler for awaitables

C) Garbage collector

D) Logging backend

Q67. Race condition is: (Select all that apply.)

A) Faster loop

B) Outcome depends on timing/interleaving of shared state access

C) Syntax warning

D) Dead code

Q68. What is the safest baseline for external HTTP call in sync code? (Select all that apply.)

A) No timeout

B) Very large retry count only

C) Explicit timeout + exception handling

D) `while True` retry

Q69. When to use `asyncio.to_thread`? (Select all that apply.)

A) For CPU SIMD kernels only

B) To offload blocking sync call from event loop

C) To replace await

D) To disable retries

Q70. Primary primitive to guard critical section: (Select all that apply.)

A) `Queue`

B) `Lock`

C) `Timer`

D) `enumerate`

Q71. `start()` vs `run()` on thread: (Select all that apply.)

A) Same behavior always

B) `start()` creates new thread; `run()` executes in current thread

C) `run()` is async

D) `start()` returns result

Q72. In sync code, what does a blocking call do? (Select all that apply.)

A) Speeds up parallelism

B) Pauses current thread progress

C) Skips error handling

D) Converts code to async

Q73. Which workload is usually a poor fit for pure sequential sync design? (Select all that apply.)

A) Single local calculation

B) Many concurrent remote calls

C) One small script

D) Deterministic pipeline

Q74. Timeout in asyncio fundamentals is usually done with: (Select all that apply.)

A) `threading.Timer`

B) `asyncio.wait_for`

C) `os.alarm` only

D) `time.sleep`

Q75. Daemon thread means: (Select all that apply.)

A) Higher priority

B) It won't keep process alive on exit

C) It is unkillable

D) It has no shared memory

Q76. `asyncio.gather` returns results in: (Select all that apply.)

A) Completion order always

B) Input order

C) Random order

D) Reverse order

Q77. Where should timeout policy live in async systems? (Select all that apply.)

A) Global only

B) At each external dependency boundary

C) Never

D) In test files only

Q78. Best design for sync service calling flaky dependency: (Select all that apply.)

A) No retry, no timeout

B) Timeout + bounded retries + fallback

C) Infinite retries

D) Catch all and ignore

Q79. If task is destroyed while pending, likely issue is: (Select all that apply.)

A) Too many dataclasses

B) Task lifecycle not tracked/awaited during shutdown

C) Typo in import

D) Invalid f-string

Q80. Why is TaskGroup preferred over loose tasks in many cases? (Select all that apply.)

A) Less typing only

B) Structured lifecycle and safer error propagation

C) It disables exceptions

D) It uses OS threads

Q81. Which metric best exposes tail-latency pain? (Select all that apply.)

A) Only average latency

B) p95/p99 latency

C) Only success count

D) CPU model name

Q82. Cancellation in asyncio is observed at: (Select all that apply.)

A) Import time

B) Await points

C) Function definition

D) Random times only

Q83. For FastAPI async handlers with blocking DB driver, practical move is: (Select all that apply.)

A) Keep blocking directly on loop

B) Use async driver or offload blocking calls to thread pool

C) Disable await

D) Use only daemon threads

Q84. For sync maintainability, prefer: (Select all that apply.)

A) One giant function

B) Small pure functions + clear boundaries

C) Hidden globals everywhere

D) Dynamic eval

Q85. Best reason to use ThreadPoolExecutor over manual thread creation: (Select all that apply.)

A) Always faster than async

B) Easier task submission, pooling, futures, lifecycle management

C) Removes need for locks

D) Converts CPU to GPU

Q86. Strong performance tuning for thread pool starts with: (Select all that apply.)

A) Max workers = 1000 always

B) Profile workload and respect external limits

C) Remove error handling

D) Disable joins

Q87. For mixed async + blocking SDK, practical approach is: (Select all that apply.)

A) Call blocking SDK directly in coroutine

B) Offload to thread executor/to_thread

C) Disable event loop

D) Remove error handling

Q88. Condition primitive is useful for: (Select all that apply.)

A) Waiting on state transition under lock

B) Replacing queue always

C) CPU vectorization

D) File compression

Q89. Why avoid sharing too much mutable state across threads? (Select all that apply.)

A) More memory savings

B) Higher race/deadlock coupling and debugging cost

C) Better determinism

D) Better cache always

Q90. On free-threaded CPython builds, interview-safe statement is: (Select all that apply.)

A) Locks are less important

B) Thread-safety discipline becomes more important

C) GIL semantics are identical

D) Threads are obsolete

Q91. Cancellation on Future usually succeeds when: (Select all that apply.)

A) Task already running long

B) Task has not started yet

C) Task completed

D) Executor is shutdown

Q92. Event primitive is best for: (Select all that apply.)

A) Protecting arithmetic

B) One-to-many readiness signaling

C) Hash lookups

D) GC tuning

Q93. Future represents: (Select all that apply.)

- **A) Finished result only**
- **B) Handle for result/error/timeout of asynchronous execution**
- **C) Lock state**
- **D) Event loop**

Q94. In interview, strongest async answer includes: (Select all that apply.)

- **A) Only syntax**
- **B) Timeout + retries + cancellation + backpressure**
- **C) Just `await` keyword**
- **D) No tradeoffs**

Q95. Graceful shutdown of worker threads should include: (Select all that apply.)

- **A) Kill process abruptly**
- **B) Stop intake, signal workers, drain queue, join**
- **C) Ignore pending tasks**
- **D) Remove timeouts**

Q96. If dependency is permanently failing, best retry posture is: (Select all that apply.)

- **A) Aggressive unbounded retry**
- **B) Bounded retries and surface failure**
- **C) Silent ignore**
- **D) Block forever**

Q97. Ignoring cancellation handling often causes: (Select all that apply.)

- **A) Better throughput**
- **B) Resource leaks / messy shutdown**
- **C) Stronger typing**
- **D) Reduced memory**

Q98. Common async code-review red flag: (Select all that apply.)

- **A) Explicit timeout per dependency**
- **B) Blocking I/O inside coroutine**
- **C) Task supervision**
- **D) Graceful shutdown**

Q99. Un-awaited coroutine warning usually means: (Select all that apply.)

- **A) Coroutine was created but never awaited/scheduled**
- **B) Loop is too fast**
- **C) GIL is disabled**
- **D) Lock not recursive**

Q100. `asyncio.shield` is used when: (Select all that apply.)

A) You want outer cancellation to skip inner critical op cancellation

B) You want to cancel everything quickly

C) You disable timeout

D) You need multiprocessing